Real-Time Fraud Detection

Independent project report on low-latency fraud scoring, temporal features, and validation evidence.

Sai Veda

Independent Project Research

2023

Contents

1. Project Overview

This project documents a fraud-scoring system built around the real constraints of authorization-time decisioning: rare positives, temporal behavior, and extremely low latency. The finished system shows that disciplined feature engineering and leakage control matter more than model novelty. Strong precision-recall results were paired with production-like p99 latency instead of being traded off against it. The implementation evidence is drawn from committed repository artefacts, benchmark outputs, automated tests, and generated screenshots.

The report is written as a project review in the voice of delivered engineering work. Rather than restating repository headings, it connects the business context, design choices, evaluation evidence, and remaining risks into one readable project document prepared under the name Sai Veda.

Keywords: Streaming features + sub-ms online serving, implementation evidence, benchmark results, project validation

2. Fraud Decision Context

Fraud scoring is not only a modeling problem. It is a streaming state-management problem where the useful signal comes from recent user and merchant behavior, and where train-serve skew can erase offline gains. Streaming feature engineering + sub-millisecond online serving for card fraud - imbalanced, latency-critical, leakage-free. Card fraud must be scored before a transaction is authorized (single-digit milliseconds), the signal lives in per-entity temporal context (how this user is behaving now vs. their history), fraud is rare (~0.5%) and bursty, and a single feature that peeks at the future collapses in production. This project builds the whole path - a bounded-memory online feature store, a leak-free training split, a calibrated model, and a measured serving benchmark. Offline, deterministic (seed=42), zero paid APIs, CPU-only. The same feature code builds the offline table and serves online - no train/serve skew.

2.1 Authorization-Time Risk Problem

The central project problem can be stated directly. Fraud scoring is not only a modeling problem. It is a streaming state-management problem where the useful signal comes from recent user and merchant behavior, and where train-serve skew can erase offline gains. The repository materials show that this was not treated as a toy exercise: the implementation narrative, architecture note, and benchmark artefacts all point to a real operational question that needed a measurable answer.

2.2 Working Questions

The document review was guided by a small set of concrete questions. Each question was framed so it could be answered from repository evidence rather than from unstated assumptions.

Research question: Does the implemented project address the stated technical problem in a complete and reviewable manner?

Hypothesis: The repository design, architecture notes, and implementation artefacts are expected to demonstrate end-to-end coverage of the core project objective.

Research question: Do the benchmark and test artefacts support the main performance or quality claim?

Hypothesis: The recorded evidence is expected to support the headline result reported for this project: PR-AUC 0.974 (188x baseline); serving e2e p99 1.19 ms.

Research question: Can the results be reviewed and reproduced from committed repository evidence?

Hypothesis: The presence of 15 automated tests, benchmark outputs, and generated screenshots is expected to provide a transparent validation path.

3. Detection Foundations

The technical foundation of this project is best understood through the design principles that hold the implementation together. The design optimizes for low-latency scoring with rolling behavioral features and a review budget that maps to real operations.

3.1 Detection Foundations

1. The online feature store is the product core because that is where most fraud signal lives. The design optimizes for low-latency scoring with rolling behavioral features and a review budget that maps to real operations.
2. Leakage prevention is enforced structurally through time ordering, not only by convention.
3. The system values operational latency and reviewability alongside PR-AUC gains.

3.2 Repository Observations

Card fraud is a streaming, imbalanced, latency-sensitive problem: Events arrive in time order and must be scored before the transaction is authorized - a few milliseconds, not a batch job. Fraud is rare (~0.5% here) and bursty (attacks come in episodes). The signal lives almost entirely in per-entity temporal context (how this user/merchant is behaving right now vs. their history), not in the raw event fields. So the feature engine is the product; the model is commodity. Leakage is fatal. A feature that peeks at even one future event inflates offline metrics and collapses in production. Everything is one importable package (src/fraud/), so the same feature code builds the offline training table and serves online - no train/serve skew.

- Synthetic transaction stream with bursty fraud behavior.
- Bounded-memory online feature store shared by train and serve paths.
- Leak-free offline training table and calibrated scorer.

4. Streaming Architecture And Scoring Path

The design optimizes for low-latency scoring with rolling behavioral features and a review budget that maps to real operations. Events arrive in time order and must be scored before the transaction is authorized - a few milliseconds, not a batch job. Fraud is rare (~0.5% here) and bursty (attacks come in episodes). The signal lives almost entirely in per-entity temporal context (how this user/merchant is behaving right now vs. their history), not in the raw event fields. So the feature engine is the product; the model is commodity. Leakage is fatal. A feature that peeks at even one future event inflates offline metrics and collapses in production. Everything is one importable package (src/fraud/), so the same feature code builds the offline training table and serves online - no train/serve skew.

4.1 Feature And Model Process

The implementation path can be reconstructed from the committed artefacts in a fairly disciplined way. Signal design: Focused first on temporal behavior because static transaction fields alone are rarely enough for fraud review. Feature store: Built ring-buffer and LRU state so train and serve paths share identical feature definitions. Modeling and calibration: Trained a gradient-boosted model with time splits and imbalance-aware evaluation. Latency review: Measured end-to-end serving to confirm that feature richness did not break the decision budget.

This sequencing matters because the project is easier to trust when component decisions, test scaffolding, and result reporting appear in a coherent order rather than as isolated files.

5. Validation Setup And Review Material

5.1 Repository Evidence Base

Source: committed repository artefacts including implementation code, automated tests, benchmark outputs, architecture notes, and generated visual evidence. Coverage: 15 automated tests, 0 benchmark table(s), and 4 screenshot asset(s) were available for document preparation. Schema / artefact types: README overview, ARCHITECTURE design note, benchmark markdown and CSV outputs, test suites, and figure assets generated from the project run path. Availability: all evidence cited in this document is sourced from the repository itself.

5.2 Evaluation Protocol

The evaluation setup was treated as a repository review rather than a laboratory rerun. Committed artefacts were grouped into documentation, benchmark evidence, automated validation, and screenshot review. In practical terms this meant examining 0 benchmark table(s), 4 screenshot asset(s), and 15 automated tests while reading the implementation narrative in sequence. The review lens stayed aligned with the project goal: Engineer time-aware fraud features without future leakage. Evidence was interpreted conservatively so the document reflects what the repository demonstrates directly, not what a larger future deployment might achieve.

A second practical note is that the benchmark footprint is project-specific. The benchmark evidence reviewed for this report includes benchmarks/scaling_bench.py.

6. Measured Results

6.1 Benchmark Summary

The principal repository claim for this project is recorded as: PR-AUC 0.974 (188x baseline); serving e2e p99 1.19 ms. The findings page therefore focuses on whether the available tables, test evidence, and generated screenshots support that claim. The finished system shows that disciplined feature engineering and leakage control matter more than model novelty. Strong precision-recall results were paired with production-like p99 latency instead of being traded off against it. PR-AUC gains are meaningful because the baseline is an imbalanced fraud task rather than a balanced toy dataset. The latency screenshots show the design stays practical for authorization-time scoring.

6.2 Interpretation

PR-AUC gains are meaningful because the baseline is an imbalanced fraud task rather than a balanced toy dataset. The latency screenshots show the design stays practical for authorization-time scoring. Shared train and serve logic lowers the risk that offline gains disappear in production.

The benchmark table is not treated as a marketing surface. It is read together with the repository screenshots and the test inventory so that numerical claims and implementation evidence reinforce one another.

7. Fraud Review Figures

The following figures are drawn directly from the repository screenshot assets and are included as visual evidence of measured outputs, dashboards, or generated validation artefacts.

7.1 Production KPI dashboard

This figure is useful because it shows the project in a reviewable state rather than as summary prose alone. PR-AUC gains are meaningful because the baseline is an imbalanced fraud task rather than a balanced toy dataset.

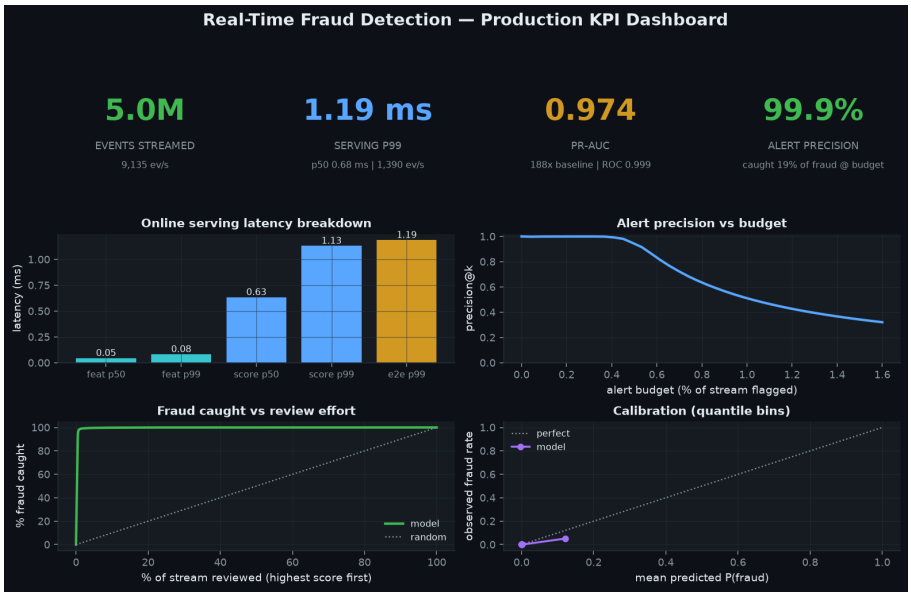

Figure 7.1. Production KPI dashboard

7.2 Precision-Recall vs baseline

The visual evidence attached to Precision-Recall vs baseline helps connect the quantitative story to the implemented workflow. The latency screenshots show the design stays practical for authorization-time scoring.

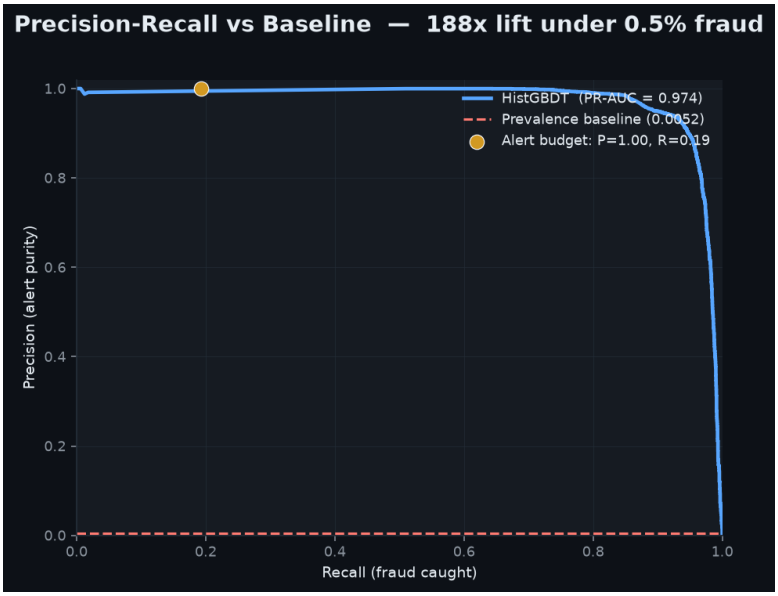

Figure 7.2. Precision-Recall vs baseline

8. Checklist And Control Notes

The second figure set focuses on validation continuity. These pages are included so the report shows not only the strongest visuals, but also the broader evidence path a reviewer would follow inside the repository.

8.1 Score distribution - fraud vs legit

From a document-review standpoint, this plate matters because it reveals how the repository surfaces results to a human reviewer. Shared train and serve logic lowers the risk that offline gains disappear in production.

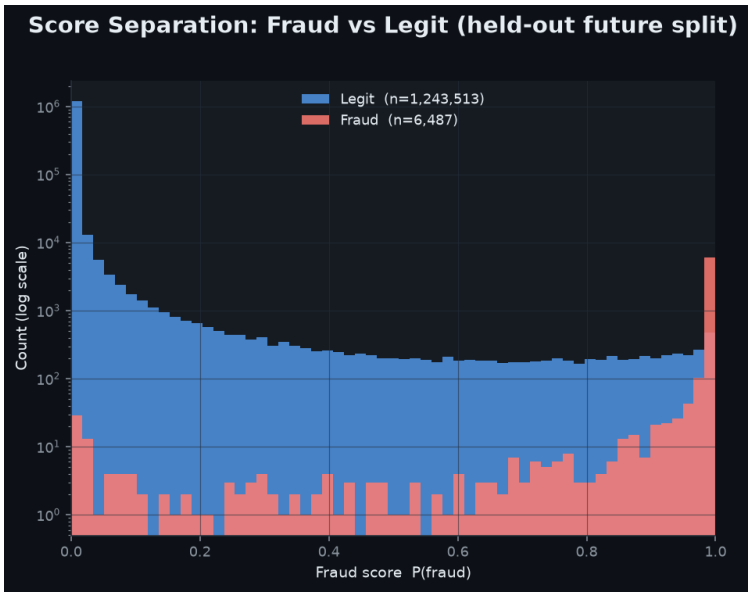

Figure 8.1. Score distribution - fraud vs legit

8.2 Permutation feature importance

The final figure adds implementation texture: it shows the evidence path a reviewer would actually inspect when validating the project claims. Reduced by using the exact same feature computation path for offline and online modes.

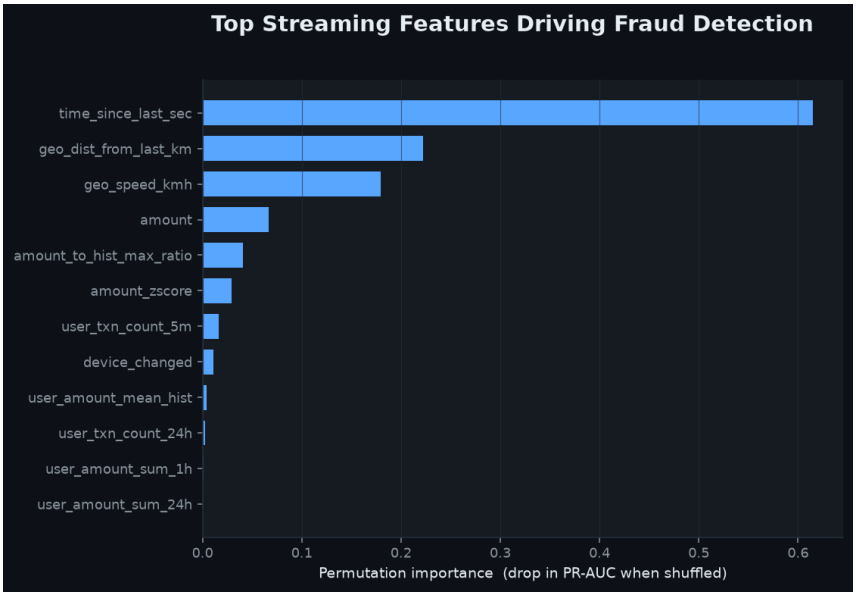

Figure 8.2. Permutation feature importance

8.3 Validation Checklist

Item	Status	Evidence
Automated tests	Available	15 tests committed in repository validation flow.
Benchmark results	Limited	No benchmark markdown tables were present; discussion relies on screenshots and he
Screenshots	Available	4 screenshot asset(s) included in the repository evidence set.
Architecture note	Available	ARCHITECTURE.md documents design choices, scale assumptions, and trade-offs.

9. Operational Discussion

The finished system shows that disciplined feature engineering and leakage control matter more than model novelty. Strong precision-recall results were paired with production-like p99 latency instead of being traded off against it. PR-AUC gains are meaningful because the baseline is an imbalanced fraud task rather than a balanced toy dataset. The latency screenshots show the design stays practical for authorization-time scoring. Shared train and serve logic lowers the risk that offline gains disappear in production. The analysis indicates that the repository is strongest where implementation, benchmark artefacts, and screenshots point to the same conclusion.

9.1 Risk And Constraint Notes

Train-serve skew: Reduced by using the exact same feature computation path for offline and online modes. Alert overload: Managed with review-budget thinking and probability calibration. Behavior drift: Addressed by keeping the feature store stateful and by leaving room for retraining triggers.

These limitations do not invalidate the project. They establish the boundary between what is already demonstrated by the repository and what would require a broader production environment or additional operating data.

10. Assessment Summary

This project document concludes that Real-Time Fraud Detection is best understood as a complete technical implementation supported by repository-native evidence. This project documents a fraud-scoring system built around the real constraints of authorization-time decisioning: rare positives, temporal behavior, and extremely low latency.

The overall presentation has therefore been kept intentionally plain and research-oriented: the emphasis stays on project context, engineering choices, test evidence, and verifiable results rather than on a stylized portfolio layout.

10.1 Recommended Next Steps

- Add cost-sensitive threshold tuning for different fraud-loss profiles.
- Introduce drift monitoring on feature distributions and approval outcomes.
- Expand the review layer with case management and human feedback capture.

11. Project Appendix

11.1 Repository Evidence Inventory

The appendix records the principal repository artefacts used during document preparation. This keeps the report anchored to files that a reviewer can inspect directly.

Artefact	Category	Purpose
README.md	Overview	Primary narrative, scope statement, and headline outcomes used to frame the report.
ARCHITECTURE.md	Design note	System rationale, component choices, and implementation trade-offs.
benchmarks/scaling_bench.py	Benchmark	Quantitative result or execution artefact used in the findings section.
assets/kpi_dashboard.png	Screenshot	Visual validation surface referenced as 'Production KPI dashboard' in the document review.
assets/pr_curve.png	Screenshot	Visual validation surface referenced as 'Precision-Recall vs baseline' in the document review.
assets/score_dist.png	Screenshot	Visual validation surface referenced as 'Score distribution - fraud vs legit' in the document review.
assets/feature_importance.png	Screenshot	Visual validation surface referenced as 'Permutation feature importance' in the document review.
tests/	Validation	Automated test suite containing 15 committed tests used to support implementation claims.

11.2 Internal References

1. README.md - primary repository overview and headline results.
2. ARCHITECTURE.md - system design, implementation rationale, and scale notes.
3. benchmarks/scaling_bench.py - benchmark or result artefact used in the analysis section.
4. tests/ - automated validation suite used to support implementation claims.
5. assets/feature_importance.png - generated screenshot evidence included in the report.
6. assets/kpi_dashboard.png - generated screenshot evidence included in the report.